

DEPLOYMENT & INTEGRATION RUNBOOK

Velocity

Complete developer documentation: every feature, function, API endpoint, background job, integration, deployment step and operational procedure

Version 1.0 · 13 September 2026 · Prepared for the engineering team

Companion artefact: [velocity-phase-4.tar.gz](#) (source code with git history). This document supersedes the phase handoff notes for day-to-day deployment and integration work.

Contents

1. How to use this document.....	4
2. System overview.....	5
2.1 What the product does.....	5
2.2 Runtime topology.....	5
2.3 Repository layout.....	5
3. Prerequisites.....	7
3.1 Tooling on the developer machine.....	7
3.2 Accounts to create before go-live.....	7
4. Source control and workflow.....	8
4.1 First push.....	8
4.2 Branching and review.....	8
4.3 Routine pull / push commands.....	8
5. Environment variables.....	9
5.1 Core (always required).....	9
5.2 Storage.....	9
5.3 Providers.....	9
5.4 Platform publishing.....	10
5.5 Billing, email, auth providers, misc.....	10
6. Database.....	11
6.1 Requirements.....	11
6.2 Commands.....	11
6.3 Schema map (37 tables).....	11
6.4 Backups.....	11
7. Local development.....	12
7.1 Fastest path (Docker, mock providers, no external keys).....	12
7.2 Without Docker.....	12
7.3 Everyday scripts.....	12
8. Deployment.....	13
8.1 Choosing a target.....	13
8.2 Render, step by step.....	13
8.3 Fly.io.....	13
8.4 Single VM.....	13
8.5 Post-deploy verification.....	14
9. Integrating the providers.....	15
9.1 Anthropic — profiles and copywriting.....	15
9.2 Firecrawl — crawler fallback.....	15
9.3 ElevenLabs — voice.....	15
9.4 Talking-head video vendor.....	15
9.5 Image generation (AI Studio and characters).....	15
9.6 TikTok.....	15
9.7 Instagram Reels (Meta).....	16
9.8 YouTube Shorts.....	16
9.9 LinkedIn.....	16
9.10 Stripe.....	16
9.11 Email.....	17
9.12 Storage and CDN.....	17
9.13 Sign-in providers (optional).....	17
9.14 Callback URL reference.....	17

10. Feature reference.....	18
10.1 Authentication and tenancy.....	18
10.2 Onboarding and the Company Profile.....	19
10.3 Generation engine.....	19
10.4 Velocity mode (swipe approval).....	20
10.5 Content library, editing and Studio.....	20
10.6 Social accounts and publishing.....	21
10.7 Automations.....	21
10.8 Analytics and attribution.....	22
10.9 Trends, characters and clips.....	22
10.10 Plans, entitlements and credits.....	23
10.11 Affiliates.....	24
10.12 Notifications and the weekly digest.....	24
10.13 Admin panel.....	24
11. REST API reference.....	26
11.1 Conventions.....	26
11.2 Status codes.....	26
11.3 Complete endpoint index.....	27
11.4 Worked example: generate and publish via the API.....	29
12. Webhooks.....	31
12.1 Verifying a signature.....	31
13. MCP server (AI agents).....	32
14. Website tracking and attribution.....	33
15. Background jobs.....	34
16. Front-end map.....	35
17. Testing and verification.....	36
17.1 Continuous integration.....	36
17.2 End-to-end API suite.....	36
17.3 Billing simulation (no Stripe account needed).....	36
17.4 Route crawl.....	36
17.5 Browser tests and screenshots.....	36
17.6 Release checklist.....	36
18. Operations.....	38
18.1 Scaling.....	38
18.2 Monitoring.....	38
18.3 Routine tasks.....	38
18.4 Incident playbook.....	38
19. Security.....	39
19.1 Controls in place.....	39
19.2 Before you go live.....	39
19.3 Key rotation.....	39
20. Extending the system.....	40
21. Troubleshooting.....	41
22. Appendix: quick command reference.....	42

1. How to use this document

This is an execution document. Sections 2–8 take an empty machine to a running production deployment. Sections 9–16 document every feature, function, endpoint and integration so a developer can extend or debug any part without reading the whole codebase. Sections 17–21 cover testing, operations and troubleshooting.

If you are...	Read
Deploying for the first time	Sections 2, 3, 5, 6, 7, 8, then 17 to verify
Wiring up a provider (Anthropic, TikTok, Stripe...)	Section 8, then the relevant feature in section 10
Building against the API	Sections 11 (REST), 12 (webhooks), 13 (MCP), 14 (tracking)
Changing a feature	Section 9 (architecture), section 10 (feature reference), section 15 (jobs)
On call	Sections 18 (operations), 19 (security), 21 (troubleshooting)

Conventions

Commands run from the repository root unless stated otherwise. \$APP is your public base URL, e.g. <https://app.example.com>.

Every API path in this document is relative to /api/v1 unless it begins with /api or /r.

"Actor" means the authenticated caller: either a browser session (cookie) or an API key (Authorization: Bearer sk_...).

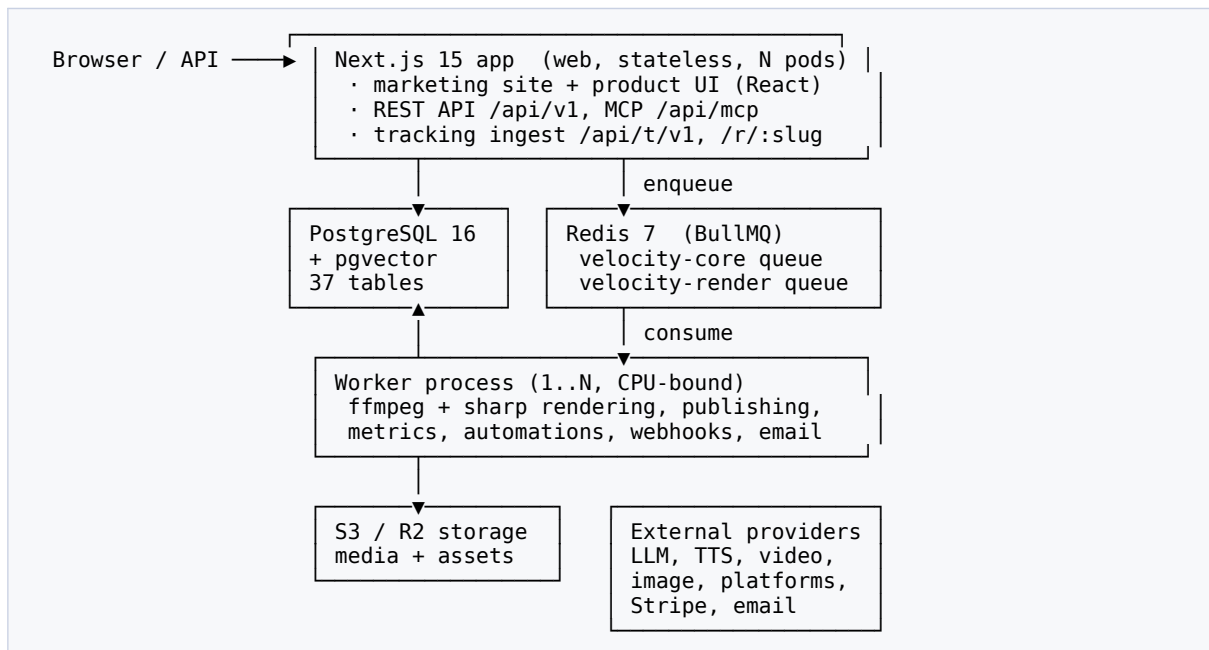
Plan gates return HTTP 402 with a machine-readable code and an upgrade_url. Treat 402 as "upsell", not "error".

2. System overview

2.1 What the product does

A user pastes a website URL. The platform crawls it, builds a Company Profile with an LLM, then generates batches of short-form content (AI UGC talking-head videos, slideshows, hook + demo clips, memes, trend remixes, human UGC). The user approves content by swiping in Velocity mode, and the platform schedules and publishes natively to TikTok, Instagram Reels, YouTube Shorts and LinkedIn, then reports analytics and website attribution. Automations keep the calendar full; an admin panel manages every account, key and integration.

2.2 Runtime topology



The worker is not optional. Generation, rendering, publishing, metrics collection, automations, webhook delivery and the weekly digest all run there. A deployment without a worker will accept content requests and never complete them.

2.3 Repository layout

```

velocity/
  src/app/(marketing)/  public site: landing, pricing, FAQ, blog,
                        27 SEO pages, legal, contact
  src/app/app/          product UI: velocity, studio, content, calendar,
                        automations, analytics, trends, characters,
                        settings/*, affiliate, admin
  src/app/api/          REST API (/api/v1), MCP, tracking, health, OG images
  src/app/r/[slug]      tracked-link redirect
  src/app/media/[...key] local storage driver
  src/db/               schema.ts (37 tables), migrate.ts, seed.ts, index.ts
  src/lib/              plans, tenancy, auth, crypto, queue, storage, llm,
                        crawler, generation, scheduling, automations,
                        stripe, webhooks, email, seo, content
                        index.ts (sharp + ffmpeg), pipeline.ts (per format)
  src/lib/render/        TikTok, Instagram, YouTube, LinkedIn, mock
  src/lib/publishers/    all 15 background jobs and their schedulers
  src/workers/index.ts   app shell, modals, media, timeline editor, pricing
  src/components/       generated SQL migrations
  drizzle/              e2e, qa, stripe-sim, backup, screenshots
  scripts/              Playwright specs
  tests/
  
```

```
.github/workflows/    ci.yml
```

3. Prerequisites

3.1 Tooling on the developer machine

Requirement	Version	Why
Node.js	22 LTS	App and worker runtime
npm	10+	Dependency management
Docker + Compose	recent	Local Postgres, Redis, MinIO; production images
ffmpeg + ffprobe	6+	Video rendering in the worker (installed in the Docker image)
Fonts: DejaVu Sans	—	Text rendering in generated slides and captions
git	2.30+	Version control
psql (optional)	16	Database inspection and restores

3.2 Accounts to create before go-live

Service	Needed for	Lead time
GitHub	Repository and CI	Minutes
Host (Render / Fly / Railway / VM)	Web + worker	Minutes
Postgres 16 with pgvector (Neon, Supabase, RDS, self-hosted)	All data	Minutes
Redis 7 (Upstash, provider add-on, self-hosted)	Job queues	Minutes
S3-compatible storage (R2, B2, S3) + CDN	Media	Minutes
Anthropic	Company Profiles and copywriting	Minutes
ElevenLabs	Voice for AI UGC	Minutes
Talking-head video vendor (HeyGen / Hedra / Sync)	AI UGC video	Hours–days
fal.ai or Replicate	AI Studio images and character references	Minutes
Stripe	Billing	Hours (account verification)
Resend (or SMTP)	Magic links, invites, digests	Minutes
TikTok for Developers	Publishing to TikTok	Weeks (app review)
Meta for Developers	Instagram Reels publishing	Weeks (App Review)
Google Cloud + YouTube Data API	Shorts publishing	Weeks (OAuth verification)
LinkedIn Developers	LinkedIn publishing	Days–weeks

Start the platform applications first

TikTok, Meta, Google and LinkedIn app reviews are the long pole — typically weeks. Submit them on day one. Everything else in this document can be completed and tested in mock mode while you wait, and each provider can be switched to live independently.

4. Source control and workflow

4.1 First push

The delivered tarball already contains an initialised repository with commit history and a .gitignore that excludes node_modules, .next, .env, .media, backups and test output.

```
tar -xzf velocity-phase-4.tar.gz && cd velocity
git log --oneline                # confirm history is present
git remote add origin git@github.com:<org>/velocity.git
git branch -M main
git push -u origin main
```

For HTTPS remotes, authenticate with a fine-grained personal access token (Contents: read/write) rather than a password.

4.2 Branching and review

1. main is protected and always deployable. Enable "Require status checks" for the CI workflow and "Require a pull request before merging".
2. Branch per change: feature/<short-name>, fix/<short-name>, chore/<short-name>.
3. Commit style: type(scope): summary — e.g. feat(publishing): add first-comment support for Instagram.
4. Open a PR. CI runs typecheck → build → migrate/seed → health check → 25-assertion e2e suite against ephemeral Postgres and Redis.
5. Squash-merge to main. The host redeploys automatically (Render/Fly/Railway autoDeploy).

4.3 Routine pull / push commands

```
git pull --rebase origin main      # take latest before starting
git checkout -b feature/my-change
npm ci                            # if package-lock.json changed
npm run typecheck && npm run build  # local gate before pushing
git add -A && git commit -m "feat(scope): summary"
git push -u origin feature/my-change # open the PR in GitHub

# after schema changes
npx drizzle-kit generate           # writes a new file to drizzle/
git add drizzle && git commit -m "chore(db): migration for <change>"
```

Never commit secrets

The .gitignore excludes .env, but run git status before the first push. If a secret is ever committed, rotate it immediately — removing it from history does not un-leak it. AUTH_SECRET, TOKEN_ENCRYPTION_KEY, STRIPE_SECRET_KEY and all platform secrets are the critical ones.

5. Environment variables

Every variable, grouped by function. Copy `.env.example` to `.env` and fill it in. The app and the worker need the same values; restart both after any change.

5.1 Core (always required)

Variable	Example	Notes
APP_URL	<code>https://app.example.com</code>	Public base URL. Used for OAuth callbacks, tracked links, MCP endpoint, OG images.
AUTH_URL	<code>https://app.example.com/api/auth</code>	Auth.js base path. <code>trustHost</code> is enabled for proxies.
AUTH_SECRET	<code>openssl rand -base64 32</code>	Session signing. Rotating it signs everyone out.
DATABASE_URL	<code>postgres://user:pw@host/db?sslmode=require</code>	Postgres 16 with the vector extension.
REDIS_URL	<code>rediss://default:pw@host:6379</code>	BullMQ queues. Persistence recommended.
TOKEN_ENCRYPTION_KEY	<code>openssl rand -hex 32</code>	AES-256-GCM key for social tokens. See the rotation warning in 19.3.
ADMIN_EMAILS	<code>you@example.com,ops@example.com</code>	Comma-separated. Grants the admin panel to those signed-in users.
NODE_ENV	<code>production</code>	Set by the platform in most cases.

5.2 Storage

Variable	Values	Notes
STORAGE_DRIVER	<code>local</code> <code>s3</code>	<code>local</code> writes to <code>.media</code> and serves <code>/media/*</code> — single node only. Use <code>s3</code> in production.
LOCAL_STORAGE_DIR	<code>./media</code>	Only for the local driver.
S3_ENDPOINT	<code>https://<acct>.r2.cloudflarestorage.com</code>	R2, B2, MinIO or AWS endpoint.
S3_BUCKET / S3_REGION	<code>velocity-media</code> / <code>auto</code>	Bucket must exist before first upload.
S3_ACCESS_KEY / S3_SECRET_KEY	—	Scoped to that bucket only.
S3_PUBLIC_URL	<code>https://media.example.com</code>	CDN or public bucket URL. Platform publishers fetch media from here, so it must be publicly reachable.

5.3 Providers

Variable	Feature it unlocks	Behaviour when unset
PROVIDER_MODE	<code>mock</code> (default) or <code>live</code>	<code>mock</code> : all providers simulated; the full loop still runs.
ANTHROPIC_API_KEY / ANTHROPIC_MODEL	Company Profiles, angles, hooks, scripts, captions	Deterministic fixture copy.
FIRECRAWL_API_KEY	Crawling JavaScript-only sites	Native fetch only; sparse sites yield

Variable	Feature it unlocks	Behaviour when unset
		thin profiles.
ELEVENLABS_API_KEY	Voiceover for AI UGC and human UGC	Silent video with burned captions.
VIDEO_PROVIDER_URL / VIDEO_PROVIDER_KEY	Talking-head AI UGC video	Captioned preview card instead of a presenter.
IMAGE_PROVIDER_URL / IMAGE_PROVIDER_KEY	AI Studio images, character reference images	Deterministic placeholder portrait.
METRICS_FIRST_PULL_DELAY_MS	Delay before the first metrics pull (default 3600000)	Set to 5000 in test environments.

5.4 Platform publishing

Variable	Where to get it
TIKTOK_CLIENT_KEY / TIKTOK_CLIENT_SECRET	TikTok for Developers → your app → Content Posting API
META_APP_ID / META_APP_SECRET	Meta for Developers → app → Settings → Basic
GOOGLE_YT_CLIENT_ID / GOOGLE_YT_CLIENT_SECRET	Google Cloud Console → APIs & Services → Credentials (OAuth client)
LINKEDIN_CLIENT_ID / LINKEDIN_CLIENT_SECRET	LinkedIn Developers → app → Auth

5.5 Billing, email, auth providers, misc

Variable	Notes
STRIPE_SECRET_KEY / STRIPE_WEBHOOK_SECRET	Secret key and the signing secret of the webhook endpoint you create in 8.10.
STRIPE_PRICE_{STARTER,GROWTH,PRO}_{MONTH,YEAR}	Six price ids. Missing ids cause checkout to throw for that plan.
RESEND_API_KEY / EMAIL_FROM	Without a key, magic links and invites print to the server log (development only).
AUTH_GOOGLE_ID / AUTH_GOOGLE_SECRET, AUTH_APPLE_ID / AUTH_APPLE_SECRET	Optional social sign-in; magic link always works.
DISCORD_INVITE_URL	Community link in the footer.
ALLOW_HTTP_WEBHOOKS	Set to 1 only in local testing; production requires https webhook URLs.
WORKER_CONCURRENCY / CORE_CONCURRENCY	Render parallelism (default 3) and core job parallelism (default 8).
DATABASE_SSL / PGBOUNCER / DB_POOL_MAX	Force TLS, PgBouncer-safe mode and pool size; auto-detected from the URL when possible.

6. Database

6.1 Requirements

PostgreSQL 16 with the vector extension available. The migration runner creates pgcrypto and vector itself, applies the generated SQL migrations, and builds the ivfflat index used for trend similarity. No manual DDL is required.

6.2 Commands

```
npm run db:migrate      # extensions + drizzle migrations + pgvector index (idempotent)
npm run db:seed         # AI model registry, 8 synthetic characters, 6 trend recipes
npx drizzle-kit generate # after editing src/db/schema.ts – commit the new drizzle/*.sql
```

render.yaml runs the migration as a pre-deploy command and fly.toml as a release command, so deployments migrate automatically. On a VM, the app container runs the migration on start.

6.3 Schema map (37 tables)

Group	Tables	Purpose
Auth	users, auth_accounts, sessions, verification_tokens	Auth.js database sessions and magic-link tokens.
Tenancy	accounts, workspaces, workspace_members	One account per customer; multiple workspaces (brands); role-based membership (admin/editor/viewer).
Brand	company_profiles (vector 1536), brand_assets	Versioned profile per workspace with an embedding; uploaded logos, screenshots, demo videos.
Libraries	characters, ugc_clips, trends, templates, ai_models	AI presenters, licensed creator clips, trend metadata + recipes, saved layouts, model registry.
Content	generation_batches, content_items, content_versions, swipe_events	Batch requests, generated items with media and provenance, edit history, approval signals.
Publishing	social_accounts, scheduled_posts, recurrences, post_metrics, account_metrics	Encrypted platform tokens, scheduled/published posts with idempotency, metric snapshots.
Automation	automations, automation_runs	Cadence rules and their execution log.
Attribution	tracking_sites, tracking_events, tracked_links	Site keys, visitor events, per-post short links.
Commerce	credit_ledger, usage_counters, affiliates, affiliate_commissions	Credit movements, plan counters, referral codes and commissions.
Platform	api_keys, webhooks, audit_log, notifications, jobs	Hashed API keys, webhook endpoints, audit trail, in-app notifications, async job status.

6.4 Backups

```
npm run backup          # pg_dump (custom format) + local media tarball, with retention
BACKUP_DIR=/var/backups BACKUP_RETENTION_DAYS=30 npm run backup

# restore
pg_restore --clean --no-owner -d "$DATABASE_URL" backups/db-<timestamp>.dump
```

If you use a managed provider, its point-in-time recovery is the primary mechanism and this script is a secondary offsite copy. Test a restore into a scratch database before you rely on either.

7. Local development

7.1 Fastest path (Docker, mock providers, no external keys)

```
cp .env.example .env          # defaults already point at the compose services
# set AUTH_SECRET to any long random string
docker compose up --build     # postgres(pgvector) + redis + minio + app + worker
open http://localhost:3000
docker compose logs -f app | grep magic-link    # click the printed sign-in link
```

Then paste any website URL on the onboarding screen. The crawler is real; copy is fixtured; slideshows, memes and hook+demo videos render locally with ffmpeg; AI UGC renders as a captioned preview card; social "connect" creates a sandbox account and "publishing" returns example permalinks with synthetic metrics.

7.2 Without Docker

```
npm ci
# provide Postgres 16 (vector extension) and Redis 7; install ffmpeg
# in .env: DATABASE_URL, REDIS_URL, STORAGE_DRIVER=local, AUTH_SECRET, APP_URL, AUTH_URL
npm run db:migrate && npm run db:seed
npm run dev          # terminal 1 – web + API on :3000
npm run worker       # terminal 2 – background jobs (mandatory)
```

7.3 Everyday scripts

Command	What it does
npm run dev / build / start	Next.js development server, production build, production server.
npm run worker	BullMQ worker for both queues. Required in every environment.
npm run typecheck	TypeScript across the whole codebase; zero errors is the merge gate.
npm run db:migrate / db:seed	Apply migrations; seed model registry, characters, trends.
npm run test:e2e	25-assertion API end-to-end run against a live stack.
npm run test:stripe	Signs synthetic Stripe events and posts them to the billing webhook.
npm run test:browser	Playwright specs (needs npx playwright install).
npm run qa	Crawls all 99 routes with a session and writes qa/index.html.
npm run backup	Database dump plus local media archive with retention.
node scripts/screenshots/shots.mjs	Renders every page in headless Chromium; viewer.mjs builds the HTML viewer.

8. Deployment

8.1 Choosing a target

Target	Web	Worker	When to choose it
Render (render.yaml)	Docker service	Background worker service	First deploy. One blueprint creates Postgres, Redis, web and worker, runs migrations pre-deploy and health-checks /api/health.
Fly.io (fly.toml)	app process group	worker process group	You want regional control or cheaper CPU for rendering.
Railway (railway.json)	Dockerfile service	Second service, start command npm run worker	Simple dashboard, plugin database and Redis.
Own VM (docker-compose.prod.yml)	app container	worker container	Full control; includes Postgres, Redis and a nightly backup container.
Vercel	Supported	NOT supported	Only if you host the worker elsewhere. ffmpeg rendering will not run in Vercel functions.

8.2 Render, step by step

1. Push the repository to GitHub (section 4.1).
2. Render dashboard → New → Blueprint → select the repository. It reads render.yaml.
3. Render creates: velocity-db (Postgres 16), velocity-redis, velocity-web (Docker) and velocity-worker.
4. Fill every environment variable marked "sync: false" in the dashboard: APP_URL, AUTH_URL, TOKEN_ENCRYPTION_KEY, ADMIN_EMAILS, S3_* and any provider keys you already have. AUTH_SECRET is generated for you; DATABASE_URL and REDIS_URL are wired automatically.
5. Deploy. The pre-deploy command runs npm run db:migrate; the health check hits /api/health.
6. Run the seed once from a Render shell: npm run db:seed.
7. Add your custom domain and update APP_URL and AUTH_URL to match, then redeploy.

8.3 Fly.io

```
fly launch --no-deploy          # reads fly.toml
fly postgres create && fly postgres attach <db>      # or point DATABASE_URL at
Neon/Supabase
fly redis create
fly secrets set AUTH_SECRET=... TOKEN_ENCRYPTION_KEY=... \
  APP_URL=https://app.example.com AUTH_URL=https://app.example.com/api/auth \
  ADMIN_EMAILS=you@example.com STORAGE_DRIVER=s3 S3_BUCKET=... S3_ACCESS_KEY=...
S3_SECRET_KEY=... \
  S3_ENDPOINT=... S3_PUBLIC_URL=...
fly deploy                    # release_command runs the migration
fly scale count app=1 worker=1 # scale worker up as rendering volume grows
fly ssh console -C "npm run db:seed"
```

8.4 Single VM

```
cp .env.example .env          # fill in, plus POSTGRES_PASSWORD
docker compose -f docker-compose.prod.yml up -d --build
docker compose -f docker-compose.prod.yml exec app npm run db:seed
# terminate TLS with Caddy/Nginx/Cloudflare Tunnel in front of 127.0.0.1:3000
```

8.5 Post-deploy verification

```
curl -s $APP/api/health      # {"status":"ok","checks":{"postgres":"ok","redis":"ok"}}  
APP_URL=$APP npm run test:e2e # 25 assertions against the live deployment  
# sign in, complete onboarding, open Admin → Integrations
```

9. Integrating the providers

Each provider is behind an interface with a mock. Configure them one at a time; anything unset keeps using its mock even when PROVIDER_MODE=live, so you can go live incrementally. Admin → Integrations shows what the running process actually sees.

9.1 Anthropic — profiles and copywriting

1. Create a key at console.anthropic.com.
2. Set ANTHROPIC_API_KEY and optionally ANTHROPIC_MODEL (default claude-sonnet-5).
3. Set PROVIDER_MODE=live and restart web + worker.
4. Verify: run onboarding on a real URL and confirm the Company Profile reflects the actual site rather than fixture text.

Where it is used: src/lib/llm.ts completeJson() with zod-validated JSON schemas, called by analyzeProfile(), generateAngles() and generateCopy() in src/lib/generation.ts. Prompts enforce tone, language and a no-medical/financial-claims rule.

9.2 Firecrawl — crawler fallback

Optional. Set FIRECRAWL_API_KEY. src/lib/crawler.ts tries native fetch across the home page plus up to 12 prioritised internal pages (pricing, features, about, FAQ...), and falls back to Firecrawl when the extracted text is under 300 characters — typical for SPA marketing sites.

9.3 ElevenLabs — voice

Set ELEVENLABS_API_KEY. src/lib/render/index.ts tts() requests eleven_multilingual_v2 with the character voice_id when present. Without a key the pipeline estimates duration at ~2.6 words/second and renders silent video with burned captions.

9.4 Talking-head video vendor

The contract is deliberately generic so you can use HeyGen, Hedra, Sync or your own service. Point VIDEO_PROVIDER_URL at an endpoint that accepts a POST and returns a video URL:

```
POST $VIDEO_PROVIDER_URL
Authorization: Bearer $VIDEO_PROVIDER_KEY
{ "script": "...", "character_image_url": "https://...", "voice_id": "...",
  "audio_base64": "...|null", "aspect": "9:16" }
→ 200 { "video_url": "https://.../out.mp4", "job_id": "optional" }
```

Implement the adapter in src/lib/render/index.ts talkingHead() if your vendor needs polling. Credits are charged at 10 per second before the call and refunded automatically if it throws.

9.5 Image generation (AI Studio and characters)

Set IMAGE_PROVIDER_URL to a fal.ai-compatible endpoint and IMAGE_PROVIDER_KEY. Request body: { prompt, image_url?, image_size, num_images }; response: { images: [{ url }] } or { image_url }. Used by generateImage() for Studio images (4 credits each), character creation and the consistency pack.

9.6 TikTok

1. Create an app at developers.tiktok.com and apply for the Content Posting API with scopes user.info.basic, video.publish, video.upload, video.list.
2. Add the redirect URI \$APP/api/v1/socials/connect/tiktok/callback.

3. Set TIKTOK_CLIENT_KEY and TIKTOK_CLIENT_SECRET.
4. Review requires a demo video of your publishing flow; record one from the calendar.
5. Until the app is approved, only accounts listed as testers can connect.

Implementation: PULL_FROM_URL publishing (so S3_PUBLIC_URL must be publicly reachable), status polling every 15s up to ~10 minutes, photo carousels via the content/init endpoint, is_aigc set from the item's AI flag, refresh-token support.

9.7 Instagram Reels (Meta)

1. Create a Meta app (Business type) and add Instagram Graph API.
2. Request instagram_basic, instagram_content_publish, instagram_manage_insights, pages_show_list, business_management in App Review.
3. Redirect URI: \$APP/api/v1/socials/connect/instagram/callback.
4. Set META_APP_ID and META_APP_SECRET.
5. The connecting user must have an Instagram Business or Creator account linked to a Facebook Page, otherwise the callback returns a clear error.

Implementation: long-lived token exchange, REELS container creation with status polling, carousel children for slideshows, optional first comment, permalink fetch, insights for plays/likes/comments/shares/saved.

9.8 YouTube Shorts

1. Google Cloud project → enable YouTube Data API v3 → create an OAuth client (Web).
2. Authorised redirect URI: \$APP/api/v1/socials/connect/youtube/callback.
3. Scopes: youtube.upload and youtube.readonly. Production use requires OAuth verification.
4. Set GOOGLE_YT_CLIENT_ID and GOOGLE_YT_CLIENT_SECRET.

Implementation: resumable upload, title truncated to 100 characters, #Shorts appended, selfDeclaredMadeForKids false, containsSyntheticMedia set for AI content, statistics endpoint for metrics, refresh tokens handled.

9.9 LinkedIn

1. Create an app at linkedin.com/developers associated with a Company Page.
2. Add the "Share on LinkedIn" and "Sign In with LinkedIn using OpenID Connect" products.
3. Redirect URI: \$APP/api/v1/socials/connect/linkedin/callback; scopes openid, profile, w_member_social.
4. Set LINKEDIN_CLIENT_ID and LINKEDIN_CLIENT_SECRET.
5. Organisation posting needs the Community Management API; pass organization_urn in platform_options once granted.

Known limitation: member-post analytics require additional partner permissions, so metrics() returns zeros for LinkedIn until you have them. Views on the calendar and analytics will read 0 for LinkedIn posts — this is a platform restriction, not a bug.

9.10 Stripe

1. Create three products (Starter, Growth, Pro) each with a monthly and a yearly price — six prices total. Yearly should be 20% below 12× monthly to match the pricing page.
2. Copy the six price ids into STRIPE_PRICE_{STARTER,GROWTH,PRO}_{MONTH,YEAR}.
3. Add a webhook endpoint at \$APP/api/v1/billing/webhook subscribed to checkout.session.completed, customer.subscription.created, customer.subscription.updated,

customer.subscription.deleted, invoice.paid and invoice.payment_failed. Copy its signing secret to STRIPE_WEBHOOK_SECRET.

4. Set STRIPE_SECRET_KEY. Enable Stripe Tax if you need automatic tax.
5. Verify without real payments: `npm run test:stripe` (see 17.3).

What each event does: subscription.created/updated → apply plan, lock workspaces above the new limit; invoice.paid → expire the previous credit balance and allocate the new monthly grant (idempotent on invoice id), and record a 30% affiliate commission if the account was referred; invoice.payment_failed → start a 7-day grace period surfaced as a banner; checkout.session.completed with a credit_pack → add pack credits; subscription.deleted → downgrade to free.

9.11 Email

Set RESEND_API_KEY and EMAIL_FROM (a verified sender). Used for magic-link sign-in, workspace invites, and the Monday 08:00 weekly performance digest (per-user opt-out via notificationPrefs.weekly_digest). Without a key, links and emails print to the server log — acceptable in development, never in production.

9.12 Storage and CDN

1. Create a bucket (R2, B2, S3) and an access key scoped to it.
2. Allow public read on objects, or put a CDN in front and set S3_PUBLIC_URL to the CDN hostname.
3. Set STORAGE_DRIVER=s3 plus the S3_* variables.
4. Confirm a rendered video URL opens in an incognito window — TikTok and Instagram fetch media from this URL, so private buckets break publishing.

9.13 Sign-in providers (optional)

Google: OAuth client with redirect \$APP/api/auth/callback/google, then AUTH_GOOGLE_ID and AUTH_GOOGLE_SECRET. Apple: Services ID with redirect \$APP/api/auth/callback/apple, then AUTH_APPLE_ID and AUTH_APPLE_SECRET. Magic-link sign-in is always available and needs no configuration beyond email.

9.14 Callback URL reference

```
$APP/api/auth/callback/google
$APP/api/auth/callback/apple
$APP/api/v1/socials/connect/tiktok/callback
$APP/api/v1/socials/connect/instagram/callback
$APP/api/v1/socials/connect/youtube/callback
$APP/api/v1/socials/connect/linkedin/callback
$APP/api/v1/billing/webhook           (Stripe)
```

Changing APP_URL later

Every callback above embeds the domain. If you move domains, update the URL in each provider console and change APP_URL and AUTH_URL together, then redeploy web and worker. Connected social accounts keep working; only new connections use the new callback.

10. Feature reference

Every feature, the functions that implement it, the endpoints it exposes and the jobs it depends on. Use this to locate code and to understand side effects before changing anything.

10.1 Authentication and tenancy

Aspect	Detail		
Sign-in	Magic link (Auth.js Nodemailer provider) plus optional Google and Apple. Database sessions, trustHost enabled for proxies.		
On first sign-in	events.createUser → ensureAccountForUser(): creates accounts row (free plan), a default workspace, an admin membership and a 10-credit allocation.		
Actor resolution	src/lib/tenancy.ts getActor(req): Bearer sk_... → hashed key lookup, plan check, Redis rate limit; otherwise session cookie. Throws 401 INVALID_API_KEY / UNAUTHENTICATED.		
Workspace access	requireWorkspace(actor, id, minRole) checks membership and role rank (viewer < editor < admin), rejects locked workspaces with 402 WORKSPACE_LOCKED, and returns { ws, role, plan }.		
Roles	admin: manage everything including members and socials. editor: create, approve, schedule. viewer: read only.		
Platform admin	users.is_platform_admin or an email in ADMIN_EMAILS. Session only — API keys are never admin.		
Audit	audit() writes account, workspace, actor, action, target and metadata for every sensitive operation.		

Method	Path	Auth / Gate	Purpose
GET	/me	Session or key	User, account, plan definition, usage (credits, saves), workspaces, admin flag, provider mode
PATCH	/me	Session	Update name, timezone, notification preferences
DELETE	/me	Session	Soft-delete the account; purged by the cleanup job after 30 days
GET	/workspaces	Session or key	List workspaces on the account
POST	/workspaces	Session · plan gate	Create a workspace (402 PLAN_LIMIT_WORKSPACES when at the cap)
GET	/workspaces/:id	Viewer	Workspace, caller role, effective plan
PATCH	/workspaces/:id	Admin	Rename, change timezone
DELETE	/workspaces/:id	Admin	Soft-delete a workspace
GET	/workspaces/:id/members	Admin	Members and pending invites
POST	/workspaces/:id/members	Admin · plan gate	Invite by email with a role; returns an invite link (also emailed)
DELETE	/workspaces/:id/members?memberId=	Admin	Remove a member or revoke an invite
POST	/invites/:token	Session	Accept an invite; returns the workspace id

10.2 Onboarding and the Company Profile

POST /workspaces/:id/profile creates a jobs row and enqueues profile.analyze. The worker crawls the site, calls the LLM against a zod schema, writes a versioned company_profiles row with a 1536-dimension embedding, renames the workspace to the detected product name, and immediately queues the first content batch. The client polls GET /jobs/:id for a three-step progress indicator.

Function	File	Responsibility
crawlSite(url, maxPages)	src/lib/crawler.ts	Fetch home + prioritised internal pages, strip markup, extract headings, images, OG image, app-store and social links; Firecrawl fallback.
analyzeProfile(crawl)	src/lib/generation.ts	LLM → Company Profile JSON: product, audience segments with pains and desires, tone do/dont, features, differentiators, competitors, pricing summary, CTA, brand colours, 8 seed hooks, content pillars, language, confidence.
profileEmbedding(profile)	src/lib/generation.ts	Vector used for trend "fit for your brand" ranking.

Method	Path	Auth / Gate	Purpose
GET	/workspaces/:id/profile	Viewer	Current profile (add ?versions=1 for history)
POST	/workspaces/:id/profile	Editor	Start crawl + analysis; returns job_id
PUT	/workspaces/:id/profile	Editor	Replace the profile manually; creates a new version and re-embeds
GET	/jobs/:jobId	Viewer of the job's workspace	Status, step, percentage, result or error

10.3 Generation engine

generate.batch produces N distinct angles (excluding the last 60 used so batches do not repeat), picks a format per item from the requested mix using a deterministic low-discrepancy sequence, assigns an eligible character (tier-gated), trend or licensed clip, then enqueues one generate.item per candidate. generate.item writes the copy, runs moderation, renders media, uploads to storage and records provenance plus a predicted score.

Format	What is produced	Renderer
ai_ugc	Talking-head video with burned word-level captions and loudness normalisation	TTS → video provider; mock renders a captioned presenter card
human_ugc	Licensed creator clip with caption beats; audio replaced or subtitles only per licence	overlayBeatsOnVideo + finalizeVideo
slideshow	5–7 branded 1080×1920 slides, exported as PNGs (TikTok photo posts) and an MP4	sharp + SVG, ffmpeg concat
hook_demo	Hook plus 3–4 on-screen beats over your uploaded demo video, or over a brand screenshot	overlayBeatsOnVideo / framesToVideo
meme	Top/bottom meme over a screenshot or brand colour, padded to 9:16	renderMeme
remix	A trend recipe's structure and timing filled with your product lines	Trend structure → caption beats
upload	Your own video or image registered as a content item	No rendering

Key functions: generateAngles(profile, n, avoid) and generateCopy(profile, angle, format, opts) in src/lib/generation.ts; renderMedia(ctx, copy) in src/lib/render/pipeline.ts; primitives renderSlide,

renderMeme, renderCaptionFrame, framesToVideo, finalizeVideo, thumbnail, scriptToSrt, tts, talkingHead, generateImage in src/lib/render/index.ts. moderateText() blocks prohibited categories before anything renders; blocked items are archived, never published.

Method	Path	Auth / Gate	Purpose
POST	/workspaces/:id/generate	Editor · daily cap	Queue a batch: count, formats, character_ids, trend_ids, language
GET	/batches/:batchId	Viewer	Batch progress and its items
POST	/content/:itemId/similar	Editor · daily cap	Generate N variants reusing angle, format, character and trend
POST	/content/:itemId/rerender	Editor	Re-render media from the current (edited) copy; returns job_id

10.4 Velocity mode (swipe approval)

GET /workspaces/:id/blitz returns the next ranked candidates and auto-refills the stack (queues 20 more) when fewer than 10 candidates are in the pipeline, respecting the daily cap. The UI prefetches the next card, supports arrow keys, K/S, Z (undo), E (edit) and C (schedule), mouse and touch, and converts any 402 into an upgrade wall.

Method	Path	Auth / Gate	Purpose
GET	/workspaces/:id/blitz	Viewer	Ranked candidates + remaining, generating, refilled, daily and save limits
POST	/content/:itemId/swipe	Editor · save quota	action: keep skip undo, with an optional inline schedule block

Side effects of keep: assertCanSave() enforces the plan save quota, the item becomes saved with a timestamp, a swipe_event is written for re-ranking, an audit entry is recorded, and a content.saved webhook is emitted. With a schedule block it also creates scheduled posts in the same request.

10.5 Content library, editing and Studio

Method	Path	Auth / Gate	Purpose
GET	/workspaces/:id/content	Viewer	Library with status/format/character/text filters and cursor pagination
GET	/content/:itemId	Viewer	Item with versions, scheduled posts and latest metrics; ?download=1 returns a signed media URL
PATCH	/content/:itemId	Editor	Edit hook, script, caption, hashtags, on-screen text, slides, meme lines (snapshots a version first)
POST	/content/:itemId?action=duplicate	Editor · save quota	Duplicate an item
DELETE	/content/:itemId	Editor	Archive (soft delete)
POST	/workspaces/:id/upload	Editor	Step 1: presign an upload (purpose: content asset)
PUT	/workspaces/:id/upload	Editor	Step 2: register the uploaded object as a brand asset or content item
GET	/workspaces/:id/assets	Viewer	Brand assets with public URLs

Method	Path	Auth / Gate	Purpose
DELETE	/workspaces/:id/assets?assetId=	Editor	Delete an asset from storage and the database
GET	/templates	Viewer	System and workspace templates
POST	/templates	Editor	Save a layout as a workspace template

Timeline editor: src/components/timeline-editor.tsx edits slides, caption beats or meme lines, PATCHes the item, then calls /render and polls the job. Upload flow: presign → PUT to storage (or to /media/:key with the local driver) → register. Brand screenshots and demo videos uploaded here are used automatically as the visual bed in slideshows and hook + demo videos.

10.6 Social accounts and publishing

Each platform implements the Publisher interface in src/lib/publishers/index.ts: authUrl, exchange, refresh, validate, publish, metrics. getPublisher(platform) returns the mock adapter whenever PROVIDER_MODE is not live, which is what makes the whole loop runnable offline.

Method	Path	Auth / Gate	Purpose
GET	/socials/connect/:platform/start?workspace_id=	Admin · plan gate	Redirects to the platform consent screen with an encrypted state
GET	/socials/connect/:platform/callback	State token	Exchanges the code, encrypts and stores tokens, upserts the account
GET	/workspaces/:id/socials	Viewer	Connected accounts with status, follower count and posting slots
PATCH	/socials/:socialId	Admin	Set posting slots (weekday + HH:MM list)
DELETE	/socials/:socialId	Admin	Disconnect; cancels that account's pending posts
POST	/schedule	Editor · scheduling gate	Schedule an item: explicit time, queue slot, or a daily/weekly recurrence
POST	/publish	Editor · scheduling gate	Publish immediately
GET	/workspaces/:id/calendar	Viewer	Posts in a date range with thumbnails and errors; ?feed=1 returns the ICS URL
GET	/workspaces/:id/calendar.ics?token=	Signed token	Subscribable calendar feed for Google/Apple Calendar
PATCH	/scheduled-posts/:postId	Editor	Reschedule, edit caption/options, approve a pending post, retry a failure
DELETE	/scheduled-posts/:postId	Editor	Cancel a post that has not published

Scheduling guarantees: scheduleItem() verifies media exists, runs each platform adapter's validate() (caption length, hashtag count, media type) and returns 422 PLATFORM_VALIDATION before anything is queued; resolves the time as explicit, next free posting slot, or now; creates a tracked short link per post; and writes an idempotency key of item+account+time so a double submit cannot double post. publish.dispatch sweeps due posts every minute; publish.post retries three times with exponential backoff, marks held when an account needs reconnecting, and notifies on final failure.

10.7 Automations

planAutomation() builds the slot plan over the horizon honouring weekdays, times, minimum spacing, per-platform daily caps and slots already taken. automation.run draws from the saved library first (or

generates the shortfall inline), then schedules per the approval mode: auto publishes without review, blitz sends candidates to Velocity mode, calendar creates pending_approval posts.

Method	Path	Auth / Gate	Purpose
GET	/workspaces/:id/automations	Viewer	List automations
POST	/workspaces/:id/automations	Editor · scheduling gate	Create: accounts, posts/day, horizon, times, weekdays, format mix, source, spacing, caps, mode, approval
GET	/automations/:autoId	Viewer	Automation with its last 20 runs
PATCH	/automations/:autoId	Editor	Rename, pause/resume, change mode, approval or config
DELETE	/automations/:autoId	Editor	Delete
POST	/automations/:autoId/preview	Viewer	Dry run: slot count per account and day, generation needed, quota impact, warnings
POST	/automations/:autoId/run	Editor · scheduling gate	Run now; returns run_id

10.8 Analytics and attribution

metrics.pull collects per-post metrics on a decaying schedule: hourly for 48 hours, then daily to 30 days, then weekly to 180. The analytics endpoint joins the latest snapshot per post and computes KPIs, breakdowns by platform, format, hour and character, 3× median outlier detection, and plain-language insight sentences.

Method	Path	Auth / Gate	Purpose
GET	/workspaces/:id/analytics	Viewer	Totals, by_platform, by_format, by_hour, by_character, top posts, outliers, insights, raw rows for CSV
GET	/workspaces/:id/tracking	Viewer	Tracking sites, snippet, attribution report by source and event, per-post rollup, tracked links
POST	/workspaces/:id/tracking	Admin	Register a domain; returns the site key and snippet
POST	/api/t/v1	Site key (public)	Event ingest (single object or array); CORS open; also usable server-side
GET	/r/:slug	Public	Tracked-link redirect with utm_source/medium/campaign and a click id

10.9 Trends, characters and clips

Method	Path	Auth / Gate	Purpose
GET	/trends	Optional session/key	Browse; for_workspace=:id ranks by cosine similarity to the Company Profile; recipes hidden on free
GET	/trends/:trendId	Paid plan	Full trend with recipe structure and timing
POST	/trends/:trendId/remix	Editor · remix gate	Generate N remixes for a workspace
GET	/characters	Session or key	System + owned characters with tier locks

Method	Path	Auth / Gate	Purpose
POST	/characters	Session · 4 credits	Create an AI influencer (generated reference image or an uploaded key)
POST	/characters/:charId/images	Owner · 4 credits/view	Consistency pack: extra reference views of your own character
GET	/ugc-clips	Session or key	Licensed human UGC clips with tier locks and monthly quota
POST	/studio/image	Editor · 4 credits/image	Generate images, optionally referencing a character; saved to the library
POST	/studio/video	Editor · 10 credits/second	Queue an AI UGC video from a script; refunded automatically on final failure

Trend library policy

trends stores metadata and a structural recipe only — never third-party media. Import is admin-only (POST /admin/trends) and upserts on (platform, external_post_id). The trends.refresh job recomputes velocity from views per day and marks records older than 90 days stale.

10.10 Plans, entitlements and credits

src/lib/plans.ts is the single source of truth: the marketing pricing table, the paywalls and the server-side gates all read the same object. Changing a limit there changes every surface at once.

Limit	Free	Starter \$29	Growth \$49	Pro \$149
Content saves	3	20	100	Unlimited
Candidates per day	20	150	400	1000
AI Studio credits / month	10 (once)	250	500	2000
Workspaces	1	1	3	10
Social accounts per platform	—	1	Unlimited	Unlimited
Scheduling & publishing	—	Yes	Yes	Yes
Trend recipes & remix	—	Yes	Yes	Yes
Human UGC clips / month	—	—	100	2000
Multi-language	—	—	—	Yes
API rate limit (req/min)	—	60	300	1000
Team invites	—	Yes	Yes	Yes

Credits: 4 per image, 10 per second of video. Monthly allocation is granted on invoice.paid and the previous balance is expired first (no rollover). Purchased packs (100/\$10, 500/\$40, 2000/\$140) are added on checkout completion. consumeCredits() throws 402 CREDITS_EXHAUSTED with the shortfall; refundCredits() reverses a charge when a provider call fails.

Method	Path	Auth / Gate	Purpose
GET	/plans	Public	Plan definitions, credit tariff, packs, yearly discount
GET	/credits	Session or key	Balance, monthly allocation, renewal date, tariff, packs, optional ledger
POST	/credits	Session or key	Estimate the cost of a job before confirming

Method	Path	Auth / Gate	Purpose
POST	/billing/checkout	Session	Stripe Checkout for a plan or a credit pack (applies directly when Stripe is unset, for development)
POST	/billing/portal	Session	Stripe Billing Portal session for cards, invoices and cancellation
POST	/billing/webhook	Stripe signature	Subscription and invoice events (see 9.10)

Downgrade behaviour: workspaces beyond the new limit are locked, not deleted; saves above the new cap stay readable but new saves are refused until the account is under the limit. This is deliberate — customers never lose work by downgrading.

10.11 Affiliates

A `?ref=code` query parameter is captured at signup into `accounts.affiliate_ref`. When a referred account pays, the Stripe webhook writes a commission at 30% of the invoice (self-referral excluded). `affiliates.settle` promotes commissions from pending to approved after a 30-day refund window; an admin marks them paid once the \$50 minimum is met.

Method	Path	Auth / Gate	Purpose
GET	/affiliate	Session	Affiliate record, link, signup and earnings stats, commission history
POST	/affiliate	Session	Join the programme with a referral code and payout email

10.12 Notifications and the weekly digest

Method	Path	Auth / Gate	Purpose
GET	/notifications	Session	In-app notifications (published, failed, reconnect needed, automation ran)
PATCH	/notifications/:id	Session	Mark as read

`digest.weekly` runs Mondays at 08:00 UTC and emails each workspace member a summary of posts published, views, likes and the top post, unless `notificationPrefs.weekly_digest` is false.

10.13 Admin panel

Session-only and gated on the `platform-admin` flag; API keys can never reach it. Ten tabs, all backed by `GET /admin?view=...` and `POST /admin`.

Tab	What it shows	Actions available
Overview	Users, plan distribution, content volume, publishing totals, credits consumed, provider mode	—
Accounts	Every account with owner, plan, workspaces, saves, credits, socials, published posts, key count, status	Search; change plan; grant credits; reset usage counters; revoke all keys; suspend; restore
Keys & webhooks	Every API key across all accounts with owner, scopes, rate limit, last used; all webhook endpoints; connected socials by platform	Revoke a key; enable/disable a webhook
Integrations	All 13 providers with configured state and their environment variable names	Links to each provider console

Tab	What it shows	Actions available	
Moderation	Content blocked by the policy classifier	Unblock and return to the stack	
Publishing	Failed and held posts with platform errors, counts by status	Diagnose; retry from the customer calendar	
Affiliates	Payable balances and full commission ledger	Mark paid (enforces the \$50 minimum)	
Costs	Per-day items, AI UGC count, credits, LLM calls, published posts, estimated spend	Tune unit costs in the admin route	
Curation	Trend import and human UGC clip registration	Bulk import trends (JSON); register a licensed clip	
Audit log	Every sensitive action with actor, target and metadata	—	
Method	Path	Auth / Gate	Purpose
GET	/admin?view=	Platform admin	overview accounts keys integrations moderation publishing affiliates costs audit
POST	/admin	Platform admin	set_plan, grant_credits, reset_usage, revoke_key, revoke_all_keys, toggle_webhook, suspend_account, restore_account, unblock_item, retire_trend, set_character_status, mark_paid
POST	/admin/trends	Platform admin	Bulk import trend metadata and recipes (upsert)
POST	/admin/ugc-clips	Platform admin	Register a licensed human UGC clip with licence terms

11. REST API reference

11.1 Conventions

Aspect	Rule
Base URL	\$APP/api/v1
Auth	Authorization: Bearer sk_live_... (or sk_test_...) for machine access; session cookie for the browser. The same handlers serve both.
Content type	application/json on request and response.
Errors	{ "error": { "code": "MACHINE_CODE", "message": "human text", ... } } with a matching HTTP status.
Plan limits	402 with a code such as PLAN_LIMIT_SAVES and an upgrade_url. Not a failure — show an upsell.
Rate limits	429 RATE_LIMITED with retry_after seconds. Per API key per minute, at the plan's rpm.
Pagination	Cursor-based where lists can grow: pass ?cursor= from next_cursor, with ?limit= (max 100).
Idempotency	Scheduling derives a key from item + account + time, so repeated submissions cannot double post.
Async work	Generation, analysis and re-render return a job_id or batch_id; poll GET /jobs/:id or GET /batches/:id.
Machine spec	GET /api/v1/openapi returns an OpenAPI 3.1 document; /developers renders the docs.

11.2 Status codes

Code	Meaning	Typical cause
200	Success	—
400 VALIDATION	Bad request	Missing or malformed field; the message names the path.
401 UNAUTHENTICATED / INVALID_API_KEY	Not signed in or bad key	Expired session, revoked key.
402 <PLAN_CODE>	Plan limit or credits	Save cap, daily candidates, scheduling gate, workspaces, API access, credits exhausted.
403 FORBIDDEN	Authenticated but not allowed	Wrong role, workspace outside the key scope, admin-only route.
404 <THING>_NOT_FOUND	Missing resource	Wrong id, or the object belongs to another account.
409 <STATE>	Wrong state	PROFILE_REQUIRED, MEDIA_NOT_READY, SOCIAL_RECONNECT, NOT_CANDIDATE, NOT_CANCELLABLE.
422 PLATFORM_VALIDATION / MODERATION	Rejected before queuing	Caption too long, wrong media type, blocked content.
429 RATE_LIMITED	Too many requests	Above the plan rpm; back off for retry_after seconds.
500 INTERNAL	Unexpected	Check app logs; the response never leaks internals.

11.3 Complete endpoint index

Sixty-seven route handlers. Grouped by area; the behaviour of each is documented in section 10.

Method	Path	Auth / Gate	Purpose
GET	/api/health	Public	Liveness: postgres and redis status, provider mode, version
GET	/api/v1/openapi	Public	OpenAPI 3.1 specification
GET	/api/og?title=	Public	Generated 1200×630 social share image
—	/api/auth/*	Public	Auth.js: sign-in, callbacks, sign-out, session, CSRF
GET	/plans	Public	Plans, credit tariff, packs
GET	/models	Public	AI model registry
GET	/me	Actor	Account, plan, usage, workspaces
PATCH	/me	Session	Update profile preferences
DELETE	/me	Session	Delete account (30-day purge)
GET	/workspaces	Actor	List workspaces
POST	/workspaces	Session	Create workspace
GET	/workspaces/:id	Viewer	Workspace detail
PATCH	/workspaces/:id	Admin	Update workspace
DELETE	/workspaces/:id	Admin	Delete workspace
GET	/workspaces/:id/members	Admin	Members and invites
POST	/workspaces/:id/members	Admin	Invite a member
DELETE	/workspaces/:id/members	Admin	Remove a member
POST	/invites/:token	Session	Accept an invite
GET	/workspaces/:id/profile	Viewer	Company Profile (+ versions)
POST	/workspaces/:id/profile	Editor	Analyse a URL (async)
PUT	/workspaces/:id/profile	Editor	Replace the profile
GET	/jobs/:jobId	Viewer	Async job status
POST	/workspaces/:id/generate	Editor	Generate a batch
GET	/batches/:batchId	Viewer	Batch progress and items
GET	/workspaces/:id/blitz	Viewer	Velocity mode candidates
POST	/content/:itemId/swipe	Editor	Keep / skip / undo (+ schedule)
GET	/workspaces/:id/content	Viewer	Content library
GET	/content/:itemId	Viewer	Item detail / signed download
PATCH	/content/:itemId	Editor	Edit copy and layout
POST	/content/:itemId	Editor	Duplicate (?action=duplicate)
DELETE	/content/:itemId	Editor	Archive

Method	Path	Auth / Gate	Purpose
POST	/content/:itemId/similar	Editor	Generate variants
POST	/content/:itemId/rerender	Editor	Re-render media
POST	/workspaces/:id/upload	Editor	Presign an upload
PUT	/workspaces/:id/upload	Editor	Register the upload
GET	/workspaces/:id/assets	Viewer	Brand assets
DELETE	/workspaces/:id/assets	Editor	Delete an asset
GET	/templates	Viewer	Templates
POST	/templates	Editor	Create a template
GET	/socials/connect/:platform/start	Admin	Begin OAuth
GET	/socials/connect/:platform/callback	State	Complete OAuth
GET	/workspaces/:id/socials	Viewer	Connected accounts
PATCH	/socials/:socialId	Admin	Posting slots
DELETE	/socials/:socialId	Admin	Disconnect
POST	/schedule	Editor	Schedule (time, queue or recurrence)
POST	/publish	Editor	Publish now
GET	/workspaces/:id/calendar	Viewer	Calendar range / ICS feed URL
GET	/workspaces/:id/calendar.ics	Token	Subscribable calendar feed
PATCH	/scheduled-posts/:postId	Editor	Reschedule / approve / retry
DELETE	/scheduled-posts/:postId	Editor	Cancel
GET	/workspaces/:id/automations	Viewer	List automations
POST	/workspaces/:id/automations	Editor	Create automation
GET	/automations/:autoId	Viewer	Automation + runs
PATCH	/automations/:autoId	Editor	Update / pause / resume
DELETE	/automations/:autoId	Editor	Delete
POST	/automations/:autoId/preview	Viewer	Dry run and quota impact
POST	/automations/:autoId/run	Editor	Run now
GET	/workspaces/:id/analytics	Viewer	KPIs, breakdowns, insights
GET	/workspaces/:id/tracking	Viewer	Attribution report and links
POST	/workspaces/:id/tracking	Admin	Register a site
POST	/api/t/v1	Site key	Tracking event ingest
GET	/r/:slug	Public	Tracked-link redirect
GET	/trends	Optional	Browse trends
GET	/trends/:trendId	Paid	Trend recipe
POST	/trends/:trendId/remix	Editor	Remix a trend

Method	Path	Auth / Gate	Purpose
GET	/characters	Actor	Character library
POST	/characters	Session	Create a character (4 credits)
POST	/characters/:charId/images	Owner	Consistency pack
GET	/ugc-clips	Actor	Licensed clip library
POST	/studio/image	Editor	Generate images
POST	/studio/video	Editor	Generate a video
GET	/credits	Actor	Balance and ledger
POST	/credits	Actor	Cost estimate
POST	/billing/checkout	Session	Checkout session
POST	/billing/portal	Session	Billing portal
POST	/billing/webhook	Stripe	Subscription and invoice events
GET	/api-keys	Actor	List keys
POST	/api-keys	Session	Create a key (shown once)
DELETE	/api-keys/:keyId	Session	Revoke a key
GET	/webhooks	Actor	List endpoints and event types
POST	/webhooks	Session	Create an endpoint (returns the signing secret once)
DELETE	/webhooks?id=	Session	Delete an endpoint
GET	/notifications	Session	Notifications
PATCH	/notifications/:id	Session	Mark read
GET	/affiliate	Session	Affiliate dashboard data
POST	/affiliate	Session	Join the programme
GET	/admin	Platform admin	Admin views
POST	/admin	Platform admin	Admin actions
POST	/admin/trends	Platform admin	Import trends
POST	/admin/ugc-clips	Platform admin	Register a clip
POST	/api/mcp	API key	MCP JSON-RPC (13 tools)
GET	/media/:key	Public	Local storage driver read
PUT	/media/:key	Editor	Local storage driver write

11.4 Worked example: generate and publish via the API

```
API=https://app.example.com/api/v1
H="Authorization: Bearer sk_live_..."
WS=$(curl -s $API/me -H "$H" | jq -r '.workspaces[0].id')

# 1. Generate 10 items
BATCH=$(curl -s -X POST $API/workspaces/$WS/generate -H "$H" -H 'content-type: application/json' \
```

```

-d '{"count":10,"formats":["ai_ugc","slideshow"]}' | jq -r .batch_id)

# 2. Poll until rendered
curl -s $API/batches/$BATCH -H "$H" | jq '.batch.status, (.items|length)'

# 3. Keep the first item
ITEM=$(curl -s "$API/workspaces/$WS/blitz?limit=1" -H "$H" | jq -r '.items[0].id')
curl -s -X POST $API/content/$ITEM/swipe -H "$H" -H 'content-type: application/json' -d
'{"action":"keep"}'

# 4. Schedule it into the next free slot on every connected account
SOC=$(curl -s $API/workspaces/$WS/socials -H "$H" | jq -c ' [.socials[]|.id]')
curl -s -X POST $API/schedule -H "$H" -H 'content-type: application/json' \
-d '{"content_item_id":"$ITEM","social_account_ids":$SOC,"queue":true}'

# 5. Watch it go out
curl -s "$API/workspaces/$WS/calendar" -H "$H" | jq '.posts[] |
{platform,status,permalink}'

```

12. Webhooks

Create endpoints in Settings → API or POST /webhooks. The signing secret is returned once. Delivery runs in the worker with five attempts and exponential backoff.

Event	Fires when	Payload data
batch.completed	A generation batch finishes	batch_id, workspace_id, completed, failed
content.saved	A user keeps an item	content_item_id, workspace_id, format, hook
post.scheduled	Posts are created for an item	content_item_id, posts[] with id, platform, scheduled_at
post.published	A post goes live	post_id, content_item_id, platform, permalink, external_post_id
post.failed	A post fails after all retries	post_id, platform, error
automation.ran	An automation run completes	automation_id, run_id, generated, scheduled, skipped

12.1 Verifying a signature

```
// header: x-velocity-signature: t=<unix>,v1=<hex>
import { createHmac, timingSafeEqual } from "crypto";

function verify(rawBody: string, header: string, secret: string) {
  const parts = Object.fromEntries(header.split(",").map((kv) => kv.split("=")));
  const expected = createHmac("sha256", secret).update(`${parts.t}.${rawBody}`).digest("hex");
  const ok = timingSafeEqual(Buffer.from(expected), Buffer.from(parts.v1));
  const fresh = Math.abs(Date.now() / 1000 - Number(parts.t)) < 300; // reject replays
  return ok && fresh;
}

// body: { id, event, created_at, data: { ... } } - deduplicate on id, respond 2xx quickly
```

13. MCP server (AI agents)

A remote Model Context Protocol server at `$APP/api/mcp` speaks JSON-RPC 2.0 over HTTP and authenticates with an API key. It lets Claude, Cursor or your own agent drive the product.

```
{
  "mcpServers": {
    "velocity": {
      "url": "https://app.example.com/api/mcp",
      "headers": { "Authorization": "Bearer sk_live_..." }
    }
  }
}
```

Tool	Purpose
list_workspaces	Workspaces on the account
get_company_profile	Profile for a workspace
generate_content	Queue a batch; returns batch_id
get_batch	Batch progress and items
list_content / get_content	Library and item detail with media URLs
list_social_accounts	Connected accounts
schedule_post / publish_now	Schedule or publish immediately
list_scheduled_posts	Upcoming and recent posts
get_analytics	Analytics for the last N days
list_trends	Trends ranked for the workspace
get_credits	Credit balance

14. Website tracking and attribution

1. In the app: Settings → Website tracking → add your domain. Copy the snippet.
2. Paste the snippet before `</head>` on every page of your marketing site.
3. Fire conversions where they happen: `velocity.track('signup')` and `velocity.track('purchase', { value: 49, currency: 'USD', order_id: '123' })`.
4. Server-side events: POST the same JSON to `$APP/api/t/v1` with your site key (useful for webhooks from your own payment system).
5. Every scheduled post gets a `/r/:slug` short link that redirects with `utm_source`, `utm_medium`, `utm_campaign` and a click id, which is what ties a purchase back to a specific post.

```
<script async src="https://app.example.com/t.js" data-site="vt_XXXXXXX"></script>
<script>
  velocity.track('signup');
  velocity.track('purchase', { value: 49, currency: 'USD', order_id: '1234' });
</script>
```

The snippet is ~1KB, uses `sendBeacon`, stores only a first-party session id and the click id, and sends no cookies. Events are retained for 13 months by default (`tracking_sites.retention_months`).

15. Background jobs

Two BullMQ queues so CPU-heavy rendering can never delay time-critical publishing. Size them with `WORKER_CONCURRENCY` (render, default 3) and `CORE_CONCURRENCY` (core, default 8).

Job	Queue	Trigger	What it does
profile.analyze	core	On demand	Crawl, build the Company Profile, queue the first batch
generate.batch	render	On demand / automation	Create angles, choose formats and assets, fan out items
generate.item	render	Fan-out	Write copy, moderate, render media, upload, score
render.item	render	Re-render / Studio video	Re-render media from current copy (user-priority)
publish.dispatch	core	Every 60s	Sweep due posts, recover posts stuck in publishing over 15 minutes
publish.post	core	Per post	Call the platform adapter, retry 3×, notify, emit webhooks, schedule metrics
metrics.pull	core	After publish, decaying	Collect metrics hourly → daily → weekly
tokens.refresh	core	Hourly	Refresh expiring tokens; mark reconnect_needed and hold posts on failure
automation.run	core	Manual / tick	Plan slots, draw or generate content, schedule per approval mode
automations.tick	core	Every 10 min	Queue due continuous automations
credits.allocate	core	invoice.paid	Expire the old balance, grant the monthly allocation (idempotent)
webhook.deliver	core	Per event	Signed POST with five attempts and backoff
affiliates.settle	core	Daily	Promote commissions to approved after 30 days
digest.weekly	core	Mon 08:00	Weekly performance email per workspace member
trends.refresh	core	Every 6h	Recompute velocity scores; mark trends over 90 days stale
cleanup	core	Daily	Archive old skipped content; purge accounts deleted over 30 days ago

16. Front-end map

Route	Purpose
/	Landing: hero, interactive swipe demo, how it works, features, testimonials, pricing, FAQ
/pricing, /faq, /blog, /blog/:slug	Pricing with comparison table, categorised FAQ with schema, blog
/tools/:slug, /compare/:slug, /alternatives/:slug, /industries/:slug	27 programmatic SEO pages with related-link blocks and share rows
/developers, /affiliates, /contact, /privacy, /terms, /careers	API docs, affiliate pitch, contact with map and directions, legal
/login, /invite/:token	Magic-link sign-in; invite acceptance
/app/onboarding	URL → profile with live progress
/app/velocity	Velocity mode: swipe, keyboard, undo, edit, schedule (/app/blitz redirects here)
/app/content, /app/content/:id	Library with filters and bulk actions; detail with versions, metrics, timeline editor
/app/studio	Targeted batches, uploads, AI Studio image and video generation
/app/calendar	Month/week/list, drag to reschedule, approve, retry, cancel, ICS export and subscribe
/app/automations, /app/analytics, /app/trends, /app/characters	Automation builder with preview; analytics with insights; trend browser; character library
/app/settings/ {profile,socials,billing,members,api,tracking }	Company profile and workspace, connections, plan and credits, team, keys and webhooks, tracking
/app/affiliate, /app/admin	Affiliate dashboard; ten-tab admin panel

Shared UI lives in `src/components`: `app-shell` (sidebar, command palette, notifications), `ui.tsx` (modals, badges, toasts, the 402 upgrade wall, `useAction` hook), `media.tsx`, `modals.tsx`, `timeline-editor.tsx`, `pricing-table.tsx`, `share.tsx`, `seo-page.tsx`.

17. Testing and verification

17.1 Continuous integration

.github/workflows/ci.yml runs on every push and pull request with Postgres (pgvector) and Redis service containers: npm ci → typecheck → build → db:migrate → db:seed → health check → test:e2e, then Playwright as a non-blocking step. Make it a required status check on main.

17.2 End-to-end API suite

```
METRICS_FIRST_PULL_DELAY_MS=5000 APP_URL=$APP npm run test:e2e
```

Creates a throwaway Growth account and API key, then asserts 25 behaviours across the whole loop: profile analysis, rendered candidates, keep/skip/undo, edit and versioning, re-render, OAuth connect, queue scheduling, publish-now, published permalink, tracked-link redirect, event ingest, attribution by platform, analytics totals, character creation with credit debit, automation preview and run, ICS feed, webhook creation, the free-plan 402 and the rate limit. Exit code is non-zero on the first failure.

17.3 Billing simulation (no Stripe account needed)

```
SIM_ACCOUNT_ID=<account uuid> npm run test:stripe -- cus_TEST1
```

Signs synthetic events with STRIPE_WEBHOOK_SECRET and posts them to the webhook, exercising plan application, workspace locking, credit allocation and expiry, dunning, credit packs, affiliate commission and signature rejection. Verify the resulting rows in accounts, workspaces, credit_ledger and affiliate_commissions.

17.4 Route crawl

```
SESSION_COOKIE="authjs.session-token=..." npm run qa # writes qa/index.html
```

Requests all 99 pages and API GET routes with a session, checks status, expected content and that every JavaScript chunk each page references loads, then writes a navigable QA console with a guided click-through order, a preview pane and a live re-check button.

17.5 Browser tests and screenshots

```
npx playwright install && npm run test:browser
node scripts/screenshots/shots.mjs && node scripts/screenshots/viewer.mjs
```

Playwright covers marketing pages, magic-link sign-in through onboarding into Velocity mode, and the free-plan save wall. The screenshot scripts render every page in headless Chromium with a signed-in session and build a single-file HTML viewer for design review.

17.6 Release checklist

#	Acceptance criterion
1	CI green on main: typecheck, build, migrations, health, e2e.
2	Migrations reviewed — new drizzle/*.sql committed and applied to staging first.
3	npm run test:e2e passes against staging with live provider keys where available.
4	npm run test:stripe passes, or a real Stripe test-mode checkout completes end to end.
5	Health endpoint green and the worker log shows "worker started" with the expected mode.

#	Acceptance criterion
6	Admin → Integrations shows every provider you expect as configured.
7	A real post publishes on at least one platform and appears in analytics within the hour.
8	Backups verified by restoring the latest dump into a scratch database.

18. Operations

18.1 Scaling

Symptom	Action
Content takes too long to render	Add worker instances or raise <code>WORKER_CONCURRENCY</code> . Rendering is CPU-bound; ~2–4 concurrent renders per 2 vCPU.
Posts publish late	Check the core queue depth and that <code>publish.dispatch</code> is running. Core work is I/O-bound; raise <code>CORE_CONCURRENCY</code> before adding machines.
Slow API under load	Scale web instances (stateless). Ensure a pooled Postgres connection string and <code>PGBOUNCER=1</code> .
Storage costs rising	Set a lifecycle rule on the bucket for archived content; the cleanup job removes database rows but not historical media by design.

18.2 Monitoring

- Uptime check on `GET /api/health` (503 when Postgres or Redis is unreachable).
- Alert on worker process absence — no health endpoint, so alert on log silence or queue depth growth.
- Admin → Publishing for failed and held posts; Admin → Costs for provider spend drift.
- Log lines are structured per job: `[core]/[render] <job> <id> done|failed`.

18.3 Routine tasks

Cadence	Task
Daily	Check failed posts and accounts needing reconnection; confirm backups ran.
Weekly	Review Admin → Costs against revenue; import fresh trends; review the moderation queue.
Monthly	Approve and pay affiliate commissions; review plan limits against actual usage; rotate any exposed keys.
Quarterly	Test a full restore; review platform API changelogs for breaking changes; re-run the Playwright suite.

18.4 Incident playbook

Incident	First steps
All publishing failing on one platform	Check Admin → Publishing for the platform error, then that platform's status page and API changelog. Token expiry shows as <code>reconnect_needed</code> ; API changes show as 4xx with a message.
Generation stalled	Check the worker is running and Redis is reachable. Inspect the render queue depth. A provider outage surfaces as repeated <code>generate.item</code> failures with the provider error.
Database at capacity	<code>tracking_events</code> is the largest table. Reduce <code>retention_months</code> , archive old rows, then scale storage.
Credits or billing wrong	Replay the Stripe event from the dashboard; <code>credits.allocate</code> is idempotent on invoice id so replays are safe.
Suspected key compromise	Admin → Keys & webhooks → revoke; rotate the provider secret; review the audit log for that account.

19. Security

19.1 Controls in place

- Social tokens encrypted at rest with AES-256-GCM (TOKEN_ENCRYPTION_KEY); API keys stored as SHA-256 hashes and shown once.
- Every data access is scoped through requireWorkspace(); API keys can be restricted to specific workspaces and scopes.
- Per-plan rate limiting in Redis; admin routes are session-only and never reachable with an API key.
- Security headers (frame, nosniff, referrer, permissions, HSTS) and poweredByHeader disabled; the Docker image runs as a non-root user.
- Stripe webhooks verified by signature; outbound webhooks signed with HMAC and a timestamp to prevent replay.
- Content passes a policy classifier before rendering; blocked items are archived and surfaced in the moderation queue.
- Audit log records every sensitive action with actor and target.

19.2 Before you go live

1. Generate fresh AUTH_SECRET and TOKEN_ENCRYPTION_KEY for production — never reuse development values.
2. Set ALLOW_HTTP_WEBHOOKS=0 and confirm all customer webhook URLs are https.
3. Restrict the storage access key to the media bucket only.
4. Replace the placeholder privacy policy, terms and the contact address with your own, reviewed by counsel.
5. Confirm ADMIN_EMAILS contains only staff accounts.

19.3 Key rotation

TOKEN_ENCRYPTION_KEY cannot be rotated casually

Every stored social token is encrypted with it. Changing the value makes existing tokens undecryptable and every connected account will need reconnecting. If you must rotate, write a migration that decrypts with the old key and re-encrypts with the new one inside a single transaction, and run it with the worker stopped.

20. Extending the system

Task	Where to start
Add a content format	Add the enum value in <code>src/db/schema.ts</code> , a brief in <code>generation.ts</code> <code>formatBrief</code> , a branch in <code>render/pipeline.ts</code> <code>renderMedia</code> , and a label in <code>src/lib/api.ts</code> <code>FORMAT_LABEL</code> .
Add a platform	Implement the Publisher interface in <code>src/lib/publishers/index.ts</code> (<code>authUrl</code> , <code>exchange</code> , <code>refresh</code> , <code>validate</code> , <code>publish</code> , <code>metrics</code>), add it to <code>getPublisher</code> and <code>PLATFORMS</code> , and add its env vars.
Change plans or limits	Edit <code>src/lib/plans.ts</code> only — pricing page, paywalls and server gates all read it. Add matching Stripe prices.
Add a webhook event	Add it to <code>WEBHOOK_EVENTS</code> in <code>src/lib/webhooks.ts</code> and call <code>emitEvent()</code> at the point it occurs.
Add a background job	Add the name to the job union in <code>src/lib/queue.ts</code> , a handler in <code>src/workers/index.ts</code> , and a scheduler entry if it is periodic.
Add an API endpoint	Create <code>src/app/api/v1/<path>/route.ts</code> using <code>route()</code> and <code>parse()</code> , enforce access with <code>requireWorkspace()</code> , then document it in <code>/api/v1/openapi</code> .
Swap an AI provider	Only <code>src/lib/llm.ts</code> , <code>render/index.ts</code> (<code>tts</code> , <code>talkingHead</code> , <code>generateImage</code>) and <code>crawler.ts</code> touch external AI services.

21. Troubleshooting

Symptom	Likely cause	Fix
Sign-in returns 500 (UntrustedHost)	AUTH_URL wrong or missing behind a proxy	Set AUTH_URL to \$APP/api/auth and APP_URL to the public origin; redeploy.
Magic link never arrives	No RESEND_API_KEY	Set the key and a verified EMAIL_FROM. In development the link prints in the app log.
Candidates stay "generating"	Worker not running, Redis unreachable, or ffmpeg missing	Check the worker process and its log; confirm ffmpeg -version inside the container.
Publishing fails with a media error	S3_PUBLIC_URL not publicly reachable	Platforms fetch media by URL. Make the bucket or CDN public and re-test in an incognito window.
TikTok/Instagram connect fails	Redirect URI mismatch or app not approved	The URI must match exactly, including scheme and trailing path. Unapproved apps only allow listed testers.
Posts stuck in "publishing"	Worker restarted mid-publish	publish.dispatch recovers them after 15 minutes; check the worker is healthy.
Analytics all zero	Metrics not pulled yet, or LinkedIn	First pull is one hour after publishing (METRICS_FIRST_PULL_DELAY_MS). LinkedIn member metrics need partner permissions.
402 on a paid account	Save cap or daily candidate cap reached	Check Admin → Accounts for usage; the error code names the exact limit.
429 from the API	Plan rate limit	Back off for retry_after seconds; upgrade the plan for a higher rpm.
pgvector errors on migrate	Extension unavailable on the instance	Use a Postgres 16 image or provider with vector; run CREATE EXTENSION vector; manually if the role lacks permission.
Webhook rejected as http	Production requires https	Use an https endpoint, or set ALLOW_HTTP_WEBHOOKS=1 for local testing only.
Credits deducted but no video	Video provider failed after the charge	The worker refunds automatically on final failure; check the credit_ledger for the refund row and the job error.

22. Appendix: quick command reference

```
# local
docker compose up --build
npm run dev / npm run worker

# full stack, mock providers
# without Docker (two terminals)

# database
npm run db:migrate && npm run db:seed
npx drizzle-kit generate
npm run backup

# after schema changes
# dump + media archive

# quality gates
npm run typecheck && npm run build
npm run test:e2e
npm run test:stripe
npm run qa
npm run test:browser

# 25 assertions, live stack
# billing state machine
# 99-route crawl → qa/index.html
# Playwright

# git
git pull --rebase origin main
git checkout -b feature/x && git push -u origin feature/x

# deploy
render: push to main (blueprint autodeploys, migrations run pre-deploy)
fly:    fly deploy && fly scale count app=1 worker=1
vm:     docker compose -f docker-compose.prod.yml up -d --build

# verify
curl -s $APP/api/health
```